ELL7285

Tree Type Network - Augmentation

Name: Archit Bansal
Date: 25 Oct 2026
Github: github.com/architb1703/Tree_Networks

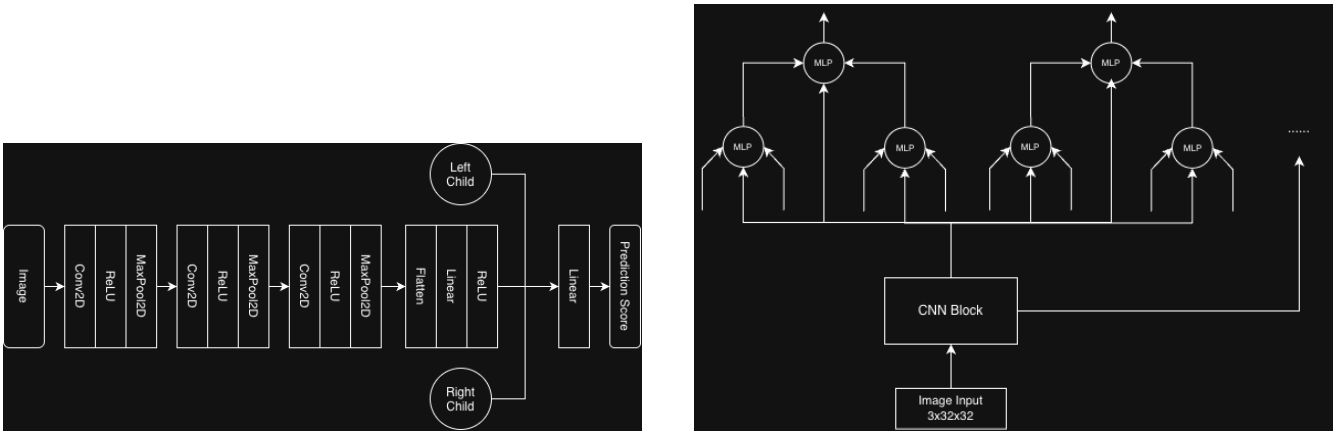
In the last assignment, the tree type network with neural network based nodes was able to overfit the CIFAR-10 training data, but exhibited very poor generalization performance on the evaluation dataset. In this assignment, I try to add augmentation and check if we can improve the generalization performance, and maybe even observe double descent on the CIFAR-10 dataset. The code and plots for this assignment are included in the github link given above, inside the "Augmentation_Experiments" folder.

1 Baseline Experiments

First, I conducted extensive experiments to verify that the poor generalization performance without augmentation does not get reduced with more parameters. The experiment details are given below:

1.1 Model Architecture

My approach to this problem was via a one-vs-all approach. As a result, for each class i, a separate tree network was trained on the binary classification task of i vs not i. Each tree node consisted of a CNN block followed by two MLP layers (Fig. 1a). In order to efficiently train the CNN block, each nodes passes its final CNN weights to the next node to provide a strong starting point for its training. The model architecture is depicted in Fig. 1b.



(a) Internal structure of node

(b) Model Architecture

Figure 1: Tree Network Architecture

1.2 Training Methodology

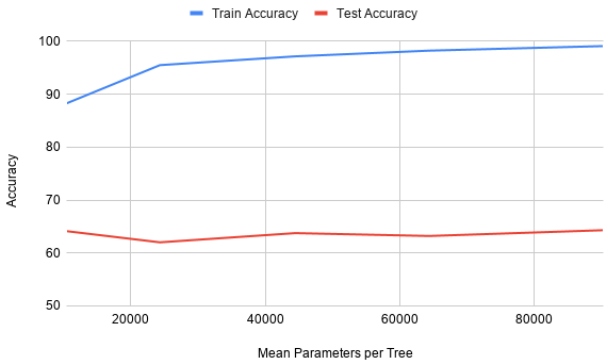
Each node is trained for 500 episodes both during the initial training phase, and during the refitting phase once the child nodes are trained. Early stopping is also implemented to stop training after 25 episodes of non-improvement, once the minimum learning rate has been hit.

1.3 Results

I conducted 5 different experiments, with increasing model sizes and compared the final training and testing multi-class classification accuracies. The CNN architecture used for each experiment are attached in the github folder. Experiment 1,2,3 used the same model architecture, but were trained with a max tree depth of 1,2 and 3 respectively. Experiments 4 and 5 had a max depth of 3, but used larger model architectures. The results are shown in Fig. 2.

Mean Parameters per Tree	Train Acc	Test Acc
10425	88.27	64.12
24325	95.49	62.01
44480	97.15	63.75
64438	98.23	63.21
90349	99.09	64.3

(a) Results



(b) Accuracy plots

Figure 2: Baseline Experiment : Train and test accuracy vs mean number of parameters per tree

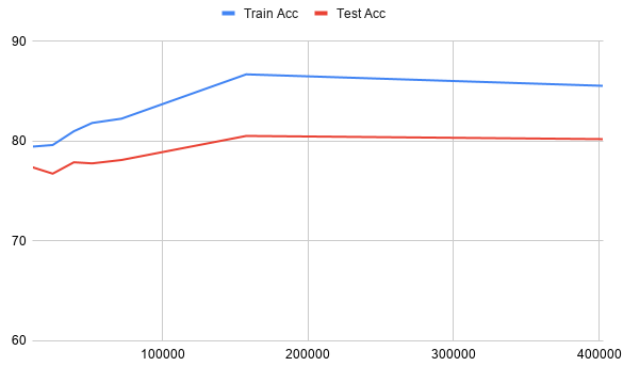
Clearly, the generalization performance for these baseline models did not improve even as we increased the parameter count.

2 Augmentation Experiments

Next, I tried adding image augmentation to the model training process. For each batch, a new augmentation operator was sampled, which was applied to all the images in that batch. After one phase of training, the original images were used to determine which samples need to be passed to the children. The training process in the children then continued again with new augmentations for each batch. The same was followed during refitting phase also. The rest of the model architecture remains the same as that used in the baseline. I also use a learning rate decay schedule that helps stabilize training by reducing the learning rate by half after every 10 episodes of no improvement on the binary cross-entropy metric. The results obtained are shown in Fig. 3. The exact architecture preferences and depth configurations for each experiment are added in the "Augmentation_SharedCNN" folder on github.

Experiment No.	Mean Parameters per Tree	Train Acc	Test Acc
1	10425	79.46	77.39
2	24325	79.62	76.75
3	38885	81	77.89
4	51430	81.83	77.78
5	71645	82.26	78.12
6	157381	86.7	80.53
7	402885	85.55	80.21

(a) Results



(b) Accuracy plots

Figure 3: Augmentation Experiment : Train and test accuracy vs mean number of parameters per tree

The generalization performance of the model improved by adding augmentation, but even after overparameterizing the model, it became impossible to overfit it on the training dataset. I was able to think to 3 reasons why this might be happening, and given the resource constraints, I tried to test two of the theories.

3 Augmentation with Common CNN Block

My first hypothesis was that the CNN blocks that each node had stored separately were highly overlapping, and therefore they gave a false sense of over-parameterization. In order to test this, I tried using a new model architecture. In this model, each tree has a single common CNN block, which is continually trained by each node as the tree grows downward till the max_depth-1 layer. Then the CNN weights are frozen, and the tree fits only the MLP layers for the leaf nodes as well as for each of the prior node's refitting phase. In order to facilitate this approach, the tree grows in a level-based traversal fashion.

Due to this architecture choice, I was able to use a larger common CNN block as well as wider MLP layers for the same number of parameters as experiment-7 in section 2. The results are shown below in Fig. 4. The exact experiment configurations are added to the "Augmentation_CommonCNN" folder on github.

CNN Block Parameters	MLP Parameters per Tree	Mean total parameters per tree	Train Acc	Test Acc
24258	231189	255717	82.86	75.85
97440	460565	558005	84.77	75.9

Figure 4: Common CNN Experiment : Train and test accuracy vs model parameters

Even with this model, I was able to overfit the training data, and the generalization accuracy also took a hit.

4 Experimenting with different augmentations

My second hypothesis regarding the results in section 2 was that I was applying too much augmentation to the dataset, because of which the models which I was able to reasonably train were not large enough to overfit it. Basically, the task was too difficult for them to learn. In order to test whether an easier augmentation scheme would benefit, I retrained the model for experiment-7 of section 2 on two simpler augmentations. The results of the experiment are shown in Fig. 5. A simpler augmentation certainly helped the model achieve better training acc while still generalizing well enough in experiment 2 from the table. This direction is worth exploring in order to achieve a overfitted model.

Augmentation Scheme	Train Acc	Test Acc
RandomFlip (p=0.5)	95.64	60.83
RandomFlip (p=0.5), RandomCrop (padding=2,constant)	92.07	79.32
RandomFlip (p=0.5), RandomCrop (padding=4,reflect), GaussianBlur	85.55	80.21

Figure 5: Comparing performance using different augmentations. Original setup from section 2 is row 3.

5 Ensemble Methods

For the ensemble method, given the limited training resources, my approach was to try and train multiple smaller tree networks for each one-vs-all task, and then take a majority vote across all the 10*n trees trained. To reduce the training time further, I also train each tree on a subset of the dataset, picking the 5000 samples of the positive class and sampling only 5000 samples from all the other negative classes. The idea was that this would also promote heterogeneity across the trees.

I was only able to run a single experiment for this architecture and it achieved a low score of 60% accuracy on train and test datasets. The experiment ran by training 3 trees per each one-vs-all task. The main reason for the

low performance was that for each data sample used in the evaluation, 2.94/3 trees on average gave the same result. This indicates that there was not enough diversity among the trees, which ended up meaning we were training 3 similar smaller networks, explaining the bad performance. Increasing the number of trees per task and increasing diversity among them are directions I would have explored next to improve performance.